

2 Stundenplaner-Projekt - Erweiterung

2.1 Kundenanforderung (Change Request)

Im ursprünglichen Projekt wurde eine zufällig zusammengesetzte Gruppe als Softwareteam eingesetzt. Die Lehrperson übernahm die Rolle des Kunden und vermittelte die Bedürfnisse und Erwartungen in Form einer mündlichen Sitzung. Das Team hatte die Aufgabe, diese Anforderungen korrekt zu erfassen, in einer geeigneten objektorientierten Sprache umzusetzen und als lauffähige, dokumentierte Lösung zu präsentieren.

Wie in den Projektvorgaben vorgesehen, kann der Kunde während und nach der Entwicklung neue Änderungswünsche einbringen. Solche Änderungen sind zu protokollieren und nachvollziehbar zu begründen.

Nach erfolgreichem Abschluss der ersten Projektphase hat der Kunde nun einen zusätzlichen Erweiterungsauftrag formuliert, der von allen Teams umzusetzen ist. Dieser Auftrag dient der Verbesserung der Softwarequalität, der Erweiterung der Funktionalität sowie der Vertiefung moderner Software-Engineering-Praktiken. Er fügt sich direkt in die bestehende Projektbeschreibung ein und baut auf den bisher entwickelten Komponenten auf.

Die folgenden Anforderungen stellen somit einen offiziellen Change Request dar, den das Team als neuen Teil des Projekts umsetzt und diese Änderungen müssen im Arbeitsrapport dokumentiert werden.

2.2 Erweiterungsauftrag

Qualitätssicherung & Architekturverbesserung

Zielsetzung

Die Erweiterungsphase soll:

- die Testbarkeit und Zuverlässigkeit der Stundenplanlogik erhöhen
- moderne Software-Engineering-Praktiken anwenden (Unit-Testing, Interfaces, Refactoring)
- die Architektur über sinnvolle Schnittstellen verbessern
- das Projekt durch Stresstests, Validierung oder GUI-Erweiterungen abrunden

Alle Umsetzungen erfolgen in **C#** und **MSTest** wird als Testframework verwendet.

2.2.1 Pflichtaufgaben (für alle Teams)

Unit-Testing mit C# und MSTest

Die Teams richten ein Testprojekt ein:

- Visual Studio → *Neues Projekt* → "MSTest Test Project"
- Der Testordner soll im Repository z. B. unter **/tests/** liegen

Pflichtumfang: mindestens 5 aussagekräftige Tests

Empfohlene Testbereiche:

1. **Bewertungskriterien testen**
 - Randzeiten werden schlechter bewertet
 - Zwischenstunden werden schlecht bewertet
2. **Ressourcenkollisionen erkennen**
 - Lehrperson doppelt zur gleichen Zeit
 - Raum doppelt belegt
 - Klasse hat zwei Fächer gleichzeitig
3. **Verfügbarkeitslogik prüfen**
 - Lehrperson ist Dienstag nachmittag blockiert → darf keinen Slot erhalten
4. **Planvalidierung**
 - Ein vollständiger Beispielplan muss als gültig erkannt werden
5. **Refactoring für Testbarkeit**
 - Wo nötig Methoden extrahieren, Logik entkoppeln
 - Keine Logik mehr direkt in der GUI oder Main()

Die Teams dokumentieren die Tests kurz (0.5 bis 1 Seite):

- Was wurde getestet?
- Was war beim Testen schwierig?
- Was wurde im Code angepasst, damit Tests möglich sind?

2.2.2 Einführung sinnvoller Interfaces in der Architektur

Viele Lösungen enthalten bisher nur konkrete Klassen wie Teacher, Room, ScheduleGenerator. Zur Verbesserung der Architektur wird die Applikation um sinnvolle Interfaces erweitert.

Mindestens zwei neue Interfaces implementieren, beispielsweise:

1. **IScheduleEvaluator**
 - bewertet einen Stundenplan nach Kriterien
 - ermöglicht Austausch oder Tests mit Mock-Klassen
2. **IAvailabilityProvider**
 - liefert Verfügbarkeiten für Lehrpersonen / Räume
 - kann für Tests stark vereinfacht oder simuliert werden
3. **IPersistenceService**
 - Laden/Speichern von JSON oder anderen Formaten
 - erleichtert Testen (Mock-Basierte Datei- oder DB-Zugriffe)
4. **IScheduleGenerator**
 - trennt Kernalgorithmus von der UI

Aufgaben

- UML-Klassendiagramm aktualisieren
- Implementierung refactoren (wo nötig)
- Bestehende Komponenten an Interfaces binden
- Mindestens ein Test verwendet einen Mock oder eine Fake-Implementierung

2.2.3 Wahlaufgaben (1 Aufgabe pro Team)

A) Algorithmus-Stresstest und Performanceanalyse

Das Projekt wird um eine vollautomatische Stresstest-Komponente ergänzt:

- Generiere 10 bis 50 zufällige Szenarien
- Messkriterien:
 - Laufzeit
 - Anzahl Konflikte
 - Plan-Bewertungen
- Ausgabe in Tabelle/JSON
- Kurze Auswertung (Flaschenhalse, Ideen zur Optimierung)

Fördert: Algorithmen, Laufzeitdenken, robuste Tests.

B) GUI-Erweiterung (WinForms oder WPF)

Für Teams ohne oder mit einfacher (textbasierter) UI:

- Stundenplan visualisieren (Tabellenansicht)
- Regel-Slider (Randzeiten, Zwischenstunden)
- Buttons: Generieren / Neu bewerten / Daten laden
- Markierungen: Konflikte farblich darstellen

Dokumentation: min. 2 Screenshots.

Fördert: GUI-Design, UX, visuelle Darstellung, Refactoring.

C) Daten-Generator und Validierungsmodul

Eine erweiterbare Komponente erstellen:

- automatische Generierung von:
 - Schüler:innen
 - Lehrpersonen
 - Räumen
 - Verfügbarkeiten
- Fehler erkennen:
 - fehlende Zuordnungen
 - widersprüchliche Verfügbarkeiten
 - ungültige Datensätze
- Erstellen eines Fehlerreports

Fördert: Datenkonsistenz, Qualitätssicherung, Testdaten.

D) Konfliktanalyse-Tool mit Erklärungsmodul

Erweiterung der Stundenplanlogik um echte Diagnostik:

- Für jeden Konflikt soll das System erklären:
 - *Warum* der Konflikt entsteht
 - *Welche Ressource* betroffen ist
 - *Welche Alternativen* möglich wären
- Konflikte in Kategorien einteilen:
 - Lehrer-Doppelbuchung
 - Raumkollision
 - Überschneidung von Klassen

- Nicht verfügbare Ressource

Format:

- Konsolenbericht oder GUI-Anzeige

Fördert: logische Analyse, Debugging, Lesbarkeit von Algorithmen.

E) Plugin-System für Bewertungsregeln (Strategy Pattern)

Neue Bewertungskriterien sollen während der Laufzeit ein- und ausschaltbar sein.

Beispiele für optionale Kriterien:

- Minimierung von Raumwechseln
- Lehrperson bevorzugt Morgenlektionen
- Klasse soll nicht mehr als 6 Lektionen pro Tag haben
- Raum soll möglichst gleichmässig genutzt werden

Aufgabenstellung:

- Implementiere ein Plugin-System:
 - Interface **IScheduleEvaluator**
 - mehrere **Evaluatoren** als einzelne Klassen
 - Konfiguration: Liste aktiver Kriterien
- Tests für mind. 2 solcher **Evaluatoren**
- UML-Anpassung mit Strategie-Pattern

Fördert: OOP-Design, Architekturqualität, Austauschbarkeit.

F) Exportmodul (Excel / PDF / ICS-Kalender)

Der Stundenplan soll exportierbar sein.

Optionen (eine wählen oder mehrere kombinieren):

- **Excel-Export**
 - Tabelle pro Klasse/Lehrperson
 - Formatierung optional
- **PDF-Export**
 - Wochenplan als Druckversion
- **ICS (Kalenderdatei)**
 - für Import in Outlook / Google Kalender

Aufgaben:

- separate Export-Komponente erstellen
- Interface: **IScheduleExporter**
- Beispiel-Exportdatei mit abgeben
- Datenformat dokumentieren

Fördert: Dateiformate, Modularität, Schnittstellenkompetenz.

Zusammenfassung der 6 Wahlaufgaben

Aufgabe	Bereich	Für Teams geeignet, die ...
A Stresstest	Algorithmen, Performance	starke Logik haben
B GUI	UI/UX	eher visuell arbeiten
C Datengenerator	Datenqualität, Tools	saubere Struktur wollen
D Konfliktanalyse	Debugging, Logik	Probleme im Algorithmus haben
E Plugin-System	Architektur, Patterns	Code verbessern möchten
F Exportmodul	Dateiformate	Praxisorientiertes Feature wollen

2.2.4 Abgabe

Im Git-Repository müssen folgenden Elemente ergänzt oder aktualisiert sein:

- **/tests/** – vollständiges MTest-Projekt
- **docs/uml/** – aktualisiertes Klassendiagramm inkl. Interfaces
- **docs/erweiterung/bericht01.pdf** oder **.md** – kurzer Erweiterungsbericht
- Implementierung der Interfaces
- Umsetzung einer Aufgabe nach Wahl (A bis F)